UPEN

Career: Engineering in Information Technologies and Digital Innovation

Subject: Object-Oriented Programming

Activity: Justification of the class diagram

Student: Alex Antonio Pinto Cataño

Delivery Date: September 14, 2026

Justification of the class diagram

Garment management system

1. The problem being modeled

The system aims to represent and manage different types of clothing items (shirts, pants, and socks). For any garment, we generally need to know basic shared characteristics, such as its size and color. Furthermore, all garments share common actions: a person can put them on, take them off, or wash them.

However, each specific type of garment has unique characteristics (for example, a shirt has a certain type of sleeve, but pants have a certain type of cut). Likewise, although all garments are washed, the washing process can vary depending on the type of garment.

This class diagram represents exactly that structure: how garments are hierarchically organized, reusing common attributes and defining specific behaviors through inheritance.

2. Why was a class diagram chosen?

Unlike other models, the class diagram shows the data structure (what attributes each entity has). It's the right tool here because the problem describes static entities, their characteristics (size, color), and their capabilities (methods), rather than the order in which events occur. It allows us to visualize key object-oriented programming concepts such as inheritance, encapsulation, and polymorphism.

3. The participants: why these and not others

- **Garment (Parent Class / Superclass): It** is modeled as a central and abstract entity because it groups all common data and behaviors. It would not make sense to repeat the attributes "size" and "color" in each individual garment; centralizing them here optimizes the model.

- **Shirt, Pants, Socks (Child Classes / Subclasses):** These are modeled as separate classes because they represent concrete objects with unique properties. A shirt needs to have its SleeveType recorded, pants its CutType, and socks their Length.

4. Structure and justification of methods and attributes

The following table explains where each element of the diagram comes from and why it has that visibility and location.

Element (Attribute/Method)	**Justification (why it exists and its design)**	**Notation used**
size, color (in Garment)	These are universal characteristics of any clothing. They are placed in the superclass so that subclasses automatically inherit them without rewriting code.	Protected attributes (#). This allows child classes to access them directly, but they remain hidden from the rest of the system.
put on(), take off() (on Garment)	These are generic actions that apply equally to any clothing item. Their implementation does not change, so they are specifically defined in the parent code.	Public methods (+), indicating that any actor or part of the system can invoke these actions.
wash() (on Garment)	All clothes get dirty and need to be washed, so laundry is essential. However, it's not possible to define how *to* wash them generically (a silk shirt is washed differently than	Abstract method (<<abstract>> and/or italics) and public (+). It forces child classes to program their own

	cotton socks).	version of the wash.
sleeveType, CutType, Length	These are unique attributes. A shirt doesn't have "length" in the same way that socks do, nor do socks have a "cut type."	Private attributes (-). They reinforce encapsulation; no one external can modify them directly.
Shirt(...), Trousers(...), Socks(...)	These are the constructor methods. They are used to create (instantiate) objects when the system starts, receiving the necessary data (size, color, and the specific attribute).	Public construction methods (+).
getTipoDeManga(), etc.	Since the specific attributes are private (-), the system needs a secure way to query or read them without altering them.	Public "getter" methods (+) that return a String type.
wash() (in subclasses)	It is the mandatory response to the father's abstract method. Each garment provides its own specific washing instructions (polymorphism).	Public methods (+).

5. Which cases does the diagram cover and which does it not (and why)

Cases that are covered:

- Code reuse through inheritance (common attributes are not repeated).

- Proper data encapsulation through visibility (+, -, #).
- The obligation to implement specific behaviors through polymorphism (wash() method).

Cases that are NOT covered (and why it's correct that they aren't):

The temporal or execution flow: This diagram does not say whether the garment is washed before or after being put on; that corresponds to a sequence or activity diagram.

Who owns the garment: The Person or Wardrobe class is not modeled. The focus is strictly on the garment hierarchy, which is valid if the exercise deals with basic inheritance.

Data validations: It is not detailed what happens if a negative size or a color that does not exist is entered.

6. How to read the notation (to explain the symbols if asked)

Rectangle divided into 3 parts = Class: The top part is the name, the middle part the attributes (variables) and the bottom part the methods (functions).

<<Abstract>> **(Stereotype):** Indicates that objects cannot be created directly from that class (you cannot instantiate a generic "Garment" in the code, it has to be a Shirt, Pants, etc.).

Sign - **= Private:** Only the class itself can see and use that element.

The # symbol **= Protected:** The class and its "children" can use that element.

The plus sign (+) **= Public:** Any part of the system can see and use that item.

Solid line with an empty triangular arrow: This is the symbol for Generalization (Inheritance). It is read from bottom to top: "A shirt *is a* garment." It always points to the superclass.

7. Possible questions from the teacher and how to answer them

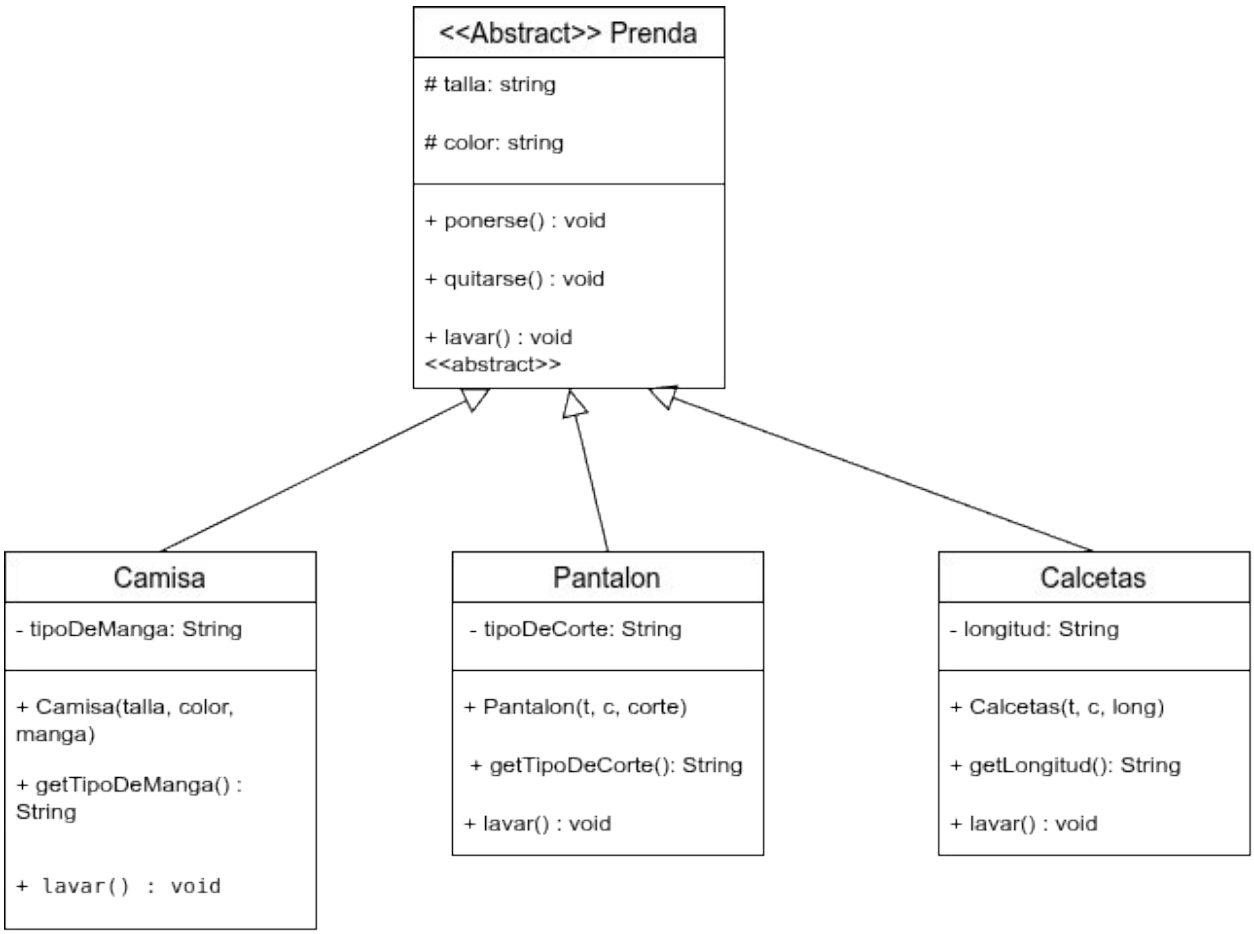
- **Q: Why is the Garment class abstract?**

- **A:** Because "Garment" is just a concept for grouping characteristics. In the real world (and in our system), you don't put on an abstract "garment," you put on a concrete physical object: a shirt or pants. Therefore, the system shouldn't allow instantiating direct objects of the Garment class.

- **Q: Why did you use protected visibility (#) for size and color, but private (-) for sleeveType?**

  - **A:** Common attributes were left protected so that child classes (Shirt, Pants) have direct access to them through inheritance without needing to create 'getter' and 'setter' methods in the parent class just for internal communication. However, the attributes specific to each subclass are private to comply with the encapsulation principle from the rest of the system, exposing them only through their 'get' methods.

- **Q: Why is `wash()` in the parent class and also in the child classes?**
  - **A:** Because it applies the concept of polymorphism. In the parent class, it's defined as an abstract meth*od* to contractually enforce that every garment can be washed. But since each fabric or garment has different washing instructions, each child class must override the method and implement its own logic.
- **Q: What does the white triangle on the lines connecting the boxes mean?**
  - **R:** Represents inheritance. It indicates that lower classes inherit all non-private attributes and methods of the class the triangle points to.



Conclusion

Creating this diagram presented some challenges that required us to carefully consider how to organize the information clearly and efficiently. During the process, we made key decisions to address the following points:

- **Avoid repeating information:** The biggest challenge was organizing garments that have things in common, but also unique details. The solution was to create a general category ("Garment") to group what they all share, such as size or color. Then, we created specific categories (Shirt, Pants, Socks) to record only the details that make them different, such as sleeve type or length.

- **Protecting the data: It** was important to decide which parts of the system could see or change the information. The general features were configured so that specific clothing types could easily use them. However, thedetalles únicos de cada

The items were hidden (making them "private") so that no other part of the system could alter or mistakenly confuse them.

- **Adapting the actions to each case:** There were actions that all garments share, such as the need to be washed, but each one is cleaned differently. Since we couldn't give a single washing instruction for everything, the solution was to establish a rule: "all garments must be washed, but each type of garment will have its own specific instructions for doing so."

- **Deciding what to include and what to leave out:** Finally, we had to limit the diagram so it wouldn't become confusing. We decided to focus solely on how the clothes are organized. We intentionally left out things like the order in which the actions happen or who owns the clothes (like a person or a wardrobe), to keep the diagram focused only on classifying the garments.